

Git Code Review

Pipeline Guide

Find and fix security problems before your code ships. A step by step guide to the sample scripts and GitHub Actions workflows, written for people who are new to pipelines.

Version 1.0.1 | November 2026

Contents

1. What this kit does
2. Words you will see
3. Before you start
4. Step 1: Make a token
5. Step 2: Store the token in GitHub
6. Step 3: Add the files to your repository
7. Step 4: Choose what the pipeline should do
8. Running the scripts on your own computer
9. Command reference
10. Security best practices
11. FAQ and troubleshooting
12. Getting help

1. What this kit does

Git Code Review scans your code on a schedule and keeps a list of what it found. This kit lets your build pipeline ask for that list, so the people who build and ship the code see the problems at the moment they can act on them.

The main tool is the fix plan. It turns the findings for one repository into a short to-do list, in the order things should be done:

Order	What it lists	What you do about it
1	Secrets in the code, such as passwords and API keys	Rotate the secret first, then remove it from the code.
2	Packages with known vulnerabilities	Upgrade to the version shown in the plan.
3	Container images with vulnerable software inside	Move to a newer base image tag and rebuild.
4	Problems in build files, such as GitHub Actions workflows and Dockerfiles	Follow the advice on the finding's page.
5	Code problems that an AI review has already read	Apply the fix the review suggests.

The kit can also:

- **Stop a deploy** when something serious is found, before it reaches your servers.
- **Attach an SBOM** (a list of every software package in the code) to each release.
- **Warn you** before your token expires, so the pipeline does not break suddenly.
- **Ask for an exception** when a finding is an accepted risk. An administrator approves it.

> **Code fixes come only from AI reviews**
>
> The plan suggests a code change only when an AI review has read that finding and the code has not changed since. Other code findings are counted, and you can open them in Git Code Review. Ask an administrator to run an AI review if you want advice on them.

2. Words you will see

Word	What it means
Pipeline	The steps that run automatically when code changes: build, test, deploy.
GitHub Actions	GitHub's pipeline service. It runs the steps for you.
Workflow	A file in `.github/workflows/` that tells GitHub Actions what to run and when.

Word	What it means
Runner	The computer that runs a workflow. GitHub provides them, or you can run your own.
Token	A long password made for a program, not a person. Ours start with `gcr_`.
Scope	What a token is allowed to do. A pipeline token needs only `reports`.
Secret (GitHub)	A value GitHub stores encrypted and hands to a workflow without showing it.
Finding	One problem the scanner found, such as a vulnerable package or a password in code.
Severity	How serious a finding is: critical, high, medium, low or info.
CVE / GHSA	The public ID of a known vulnerability, such as CVE-2021-23337.
Known exploited	Attackers are already using this vulnerability. Fix these first.
SBOM	Software Bill of Materials: a list of every package in the code and its version.
Exit code	A number a script ends with. 0 means OK. Anything else stops the pipeline.

3. Before you start

Check each of these. Most problems people hit come from one of them.

You need	How to check	If you do not have it
An account on Git Code Review that can see your repository	Sign in and find the repository on the Repositories page.	Ask an administrator to give you access.
Permission to change your repository's settings in GitHub	You can open Settings in the repository.	Ask the repository owner, or have them do steps 2 and 3.
A network path from the runner to Git Code Review	See the FAQ: "HTTP 403: refused before it reached the API".	Ask an administrator to allow the runner's IP address.
Your repository is being scanned	It has findings, or a last scanned date, in Git Code Review.	Ask an administrator to add it, and the branch you deploy, to a scan.

GitHub's own runners and IP allow lists

If your server only accepts known IP addresses, GitHub's hosted runners (`runs-on: ubuntu-latest`) will usually be refused. They come from thousands of addresses that change all the time.

Use a self-hosted runner inside your network, or GitHub's larger runners with a static IP range, and ask an administrator to allow that address. The workflows in this kit read the runner to use from a variable named `GCR_RUNNER`, so you can switch without editing them.

4. Step 1: Make a token

1. Sign in to Git Code Review.
2. Click IDE tokens in the menu on the left.
3. In the "Make one" box, type a name that says where it will be used, for example `payments repository pipeline`.
4. Set "Expires after" to 90 days or less.
5. Tick only `reports`. Untick everything else.
6. Click "Make the token".
7. Copy the token straight away. It is shown once and never again.

What each scope allows:

Scope	What it is for	Give it to
reports	Reads your findings, fix plan, SBOMs and trends, runs the deploy check, and asks for exceptions. Sees only repositories your account can open.	Build and deploy pipelines. This kit uses only this one.
review	Sends code to be checked and gets problems back. Nothing is saved.	Editor plugins.
rules	Reads the triage rules, so an editor hides the same noise.	Editor plugins.
gate	Runs the pull request gate, which scans only what a pull request changed.	Pull request checks.

Treat the token like a password

Never paste it into a workflow file, a script, a chat message or a ticket. Anybody who has it can read your findings.

If it is ever exposed, revoke it on the IDE tokens page at once and make a new one.

5. Step 2: Store the token in GitHub

GitHub keeps secrets encrypted and hands them to your workflow without ever showing them in the logs.

1. Open your repository on GitHub.
2. Click Settings (at the top of the repository, not your profile settings).
3. In the left menu, click Secrets and variables, then Actions.
4. On the Secrets tab, click New repository secret. Name: `GCR_TOKEN`. Secret: paste the token. Click Add secret.
5. On the Variables tab, click New repository variable. Name: `GCR_URL`. Value: your server's API address. It is in the kit's `.env.example` file, and looks like `https://gcr.example.com/api/v1`. Click Add variable.
6. Optional: add a variable `GCR_RUNNER` with the label of your self-hosted runner, such as `self-hosted`.

> **Many repositories?**
> An organization owner can add `GCR_TOKEN` and `GCR_URL` once under the organization's settings and share them with chosen repositories. Use a separate token for each team, so one can be revoked without breaking the others.

6. Step 3: Add the files to your repository

Unzip the kit. Copy two folders and one workflow into your repository, so it looks like this:

```
your-repository/
  .github/
    gcr/
      bash/          <- copy the kit's bash folder here
      python/        <- copy the kit's python folder here
    workflows/
      gcr-fix-plan.yml   <- copy one or more files from github-actions/
```

On a computer with Git, from the folder where you unzipped the kit:

Copy the files and push them

```
# Change this to where your repository is
REPO=~/code/your-repository

mkdir -p "$REPO/.github/gcr" "$REPO/.github/workflows"
cp -r gcr-pipeline-samples/bash gcr-pipeline-samples/python "$REPO/.github/gcr/"
cp gcr-pipeline-samples/github-actions/gcr-fix-plan.yml "$REPO/.github/workflows/"

cd "$REPO"
git add .github
git commit -m "Add Git Code Review checks"
git push
```

Then open the Actions tab of your repository on GitHub. You will see the workflow run. Click it, and the fix plan is on the Summary page.

7. Step 4: Choose what the pipeline should do

Each workflow in the kit does one job. Use any of them, alone or together.

7.1 See a fix plan every week and on every pull request

File: `github-actions/gcr-fix-plan.yml`. It never fails a build because of findings. It shows the to-do list on the run's Summary page, and keeps it as a download. It also warns when the token has less than 14 days left.

What the Summary page shows (shortened):

```
## Fix plan for `acme/payments` on `main`

### 1. Secrets in the code: do these first
| Severity | What                | Where              | Live?                  |
| high     | AWS key AKIA****WXYZ | src/config.js:3    | YES, confirmed working |

### 2. Packages to upgrade
| Package       | Now     | Upgrade to | Worst | Declared in       |
| lodash (npm)  | 4.17.20 | 4.17.21    | high  | package-lock.json |

### 3. Container images to rebuild
- node:18 (critical): openssl 3.0.0

### 4. Problems in the build and CI files
- medium .github/workflows/build.yml:14 unpinned action reference.

### 5. Code fixes reviewed by AI
- high src/report.py:42 SQL built from request input
  - Fix: Pass month as a bound parameter to cursor.execute.
  - Until then: Give the report user read access only.
```

On a pull request, the plan is for the branch being merged into, because that is the branch the scanner has read.

7.2 Stop a deploy when something serious is found

File: `github-actions/gcr-deploy-gate.yml`. The check job runs first, and the deploy job only runs if it passes. To add the check to a workflow you already have, copy the `security-check` job and add `needs: security-check` to your deploy job.

The part to copy

```
  security-check:
    runs-on: ${{ vars.GCR_RUNNER || 'ubuntu-latest' }}
    steps:
      - uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  # v4.2.2
        with:
          persist-credentials: false
      - name: Check for anything to fix
        env:
          GCR_URL: ${{ vars.GCR_URL }}
          GCR_TOKEN: ${{ secrets.GCR_TOKEN }}
        run: bash .github/gcr/bash/gcr-deploy-check.sh --fail-on high

  deploy:
    needs: security-check
```

The check answers one of three ways:

Answer	Exit code	What happens
PASS	0	Nothing at or above the level you chose. The deploy goes ahead.
FAIL	1	Something must be fixed. The run lists each item, with the fix, and marks the file it is in.
UNKNOWN	2	That branch was never scanned, so there is nothing to check. The deploy stops. See the FAQ.

To change how strict it is, change `--fail-on high`:

- **--fail-on critical** stops only for critical findings. A good place to start.
- **--fail-on high** stops for high and critical. The usual choice.
- **--warn-only** reports everything but never stops the deploy. Good for a trial week.

> **Known exploited packages always stop the deploy**
> A package attackers are already using stops the deploy at any severity. Add `--no-block-exploited` (Python) only if you have a strong reason.

7.3 Attach an SBOM to every release

File: `github-actions/gcr-release-sbom.yml`. When you publish a release on GitHub, it downloads the SBOM in two standard formats, CycloneDX and SPDX, and attaches both to the release. Customers and auditors often ask for these.

7.4 Ask for an exception from a script

When a finding is an accepted risk, for example a tool that never runs in production, you can ask for an exception. It waits for an administrator. Nothing changes until they approve it in Git Code Review.

Ask for an exception on finding 4182

```
bash bash/gcr-request-exception.sh \
  -i 4182 \
  -e 2027-01-31 \
  -m "Only used by the build, never shipped to servers." \
  -t SEC-1203
```

The finding number is the `id` in the fix plan's details link, or in the address of the finding's page.

7.5 Fail when things get worse

`bash/gcr-trend-guard.sh` fails when a severity has gone up over a period. Run it weekly to catch a team falling behind.

```
bash .github/gcr/bash/gcr-trend-guard.sh --days 30 --severity critical
```

8. Running the scripts on your own computer

Useful for trying things out before adding them to a pipeline. You need Python 3.8 or newer for the Python script, or `curl` and `jq` for the bash scripts. Windows users: use WSL (Windows Subsystem for Linux) or Git Bash for the bash scripts.

8.1 Set up the .env file

The `.env` file holds your settings. It is the standard way to keep settings out of code. The kit's `.gitignore` already stops it from being committed.

From the unzipped kit folder

```
cd gcr-pipeline-samples
cp .env.example .env
chmod 600 .env        # only you can read it
```

8.2 Keep the token encrypted on disk

The safest way to keep the token on your computer is encrypted, so a copy of your files or a backup does not expose it. The kit includes a helper that does this with GPG, a standard encryption tool.

Save the token encrypted

```
bash bash/gcr-token-store.sh save
# Paste your Git Code Review token (it will not be shown):
# Choose a passphrase to protect it:
# Saved to /home/you/.config/gcr/token.gpg
# Add this line to your .env file:
#   GCR_TOKEN_COMMAND=gpg --quiet --decrypt /home/you/.config/gcr/token.gpg
```

Add the line it prints to `.env`. When a script needs the token, it runs that command, and GPG asks for your passphrase. GPG remembers it for a while, so you are not asked every time.

If you prefer your computer's own password store, use one of these in `.env` instead:

System	Save the token once	Line for .env
macOS Keychain	`security add-generic-password -s gcr-token -a $USER -w`	`GCR_TOKEN_COMMAND=security find-generic-password -w -s gcr-token`
Linux (GNOME, KDE)	`secret-tool store --label=gcr service gcr-token`	`GCR_TOKEN_COMMAND=secret-tool lookup service gcr-token`
1Password CLI	Save it in 1Password as usual	`GCR_TOKEN_COMMAND=op read op://Private/gcr/token`

Avoid putting the token itself in .env

`GCR_TOKEN=` in `.env` works, but the token then sits on disk in plain text. Use it only on a machine nobody else uses. The scripts refuse a `.env` holding a token if other users can read it.

8.3 Try it

```
# Is the token working?
python3 python/gcr_api.py ping

# What should I fix in this repository?
python3 python/gcr_api.py fix-plan --repo your-org/your-repo --branch main

# Would a deploy pass?
bash bash/gcr-deploy-check.sh -r your-org/your-repo -b main
```

9. Command reference

Every feature is in the Python script. The bash scripts cover the same ground for places without Python. Leave out `--repo` and `--branch` inside GitHub Actions, and they are filled in for you.

Python: gcr_api.py ...	Bash script	What it does
ping	gcr-token-check.sh	Checks the token works and shows when it expires.
token-expiry --warn-days 14	gcr-token-check.sh -w 14	Warns when the token expires soon.
fix-plan	gcr-fix-plan.sh	The to-do list: secrets, upgrades, images, build files, AI code fixes.
check --fail-on high	gcr-deploy-check.sh -f high	Pass, fail or unknown for a deploy.
findings --format csv	gcr-findings-summary.sh --csv f.csv	Lists open findings, or saves them for a spreadsheet.
sbom --format cdx	gcr-sbom.sh -f cdx	Downloads the SBOM (cdx or spdx).

Python: gcr_api.py ...	Bash script	What it does
trends --days 30	gcr-trend-guard.sh -d 30	Shows how the numbers moved; can fail when they rise.
exceptions		Lists the exceptions you asked for.
request-exception	gcr-request-exception.sh	Asks for an exception on one finding.
	gcr-token-store.sh save	Saves your token encrypted on your computer.

Add `--help` to any command to see every option, for example `python3 python/gcr_api.py check --help`.

Exit codes

Code	Meaning
0	Worked, or the check passed.
1	There is something to fix (the check failed, or a limit was passed).
2	Unknown: the branch was never scanned.
3	The call itself failed: no token, no network, or a wrong setting. The message says which.

10. Security best practices

This kit follows these rules, and your pipelines should too.

Practice	Why it matters	How the kit helps
Least privilege	A leaked token can only do what its scope allows.	Pipelines need only `reports`. It cannot approve, change settings or send code.
One token per pipeline	You can revoke one without breaking the rest.	Name each token after where it is used.
Short expiry, then rotate	An old forgotten token is a common way in.	`token-expiry` warns 14 days before.
Secrets in a secret store	Files get copied, backed up and committed.	GitHub secrets in CI. Encrypted file or keychain on your computer.
Never commit settings	Git history keeps everything forever.	`.gitignore` excludes `.env`, token files and `.gpg` files.

Practice	Why it matters	How the kit helps
Private files	Other users of a shared machine must not read your token.	The scripts refuse a token file or `.env` others can read (`chmod 600`).
Encrypt in transit	A token sent over plain HTTP can be read on the network.	The scripts refuse `http://` and always check the server certificate.
Keep tokens off command lines	Other users can see running commands' arguments.	The token is passed to `curl` through a file descriptor, never as an argument.
Pin what you run	A changed action could run anything in your pipeline.	The workflows pin each action to an exact commit.
Read-only workflow permissions	Limits what a compromised step can change.	`permissions: contents: read`, except the release job, which must attach files.
Check what you download	Confirms the kit is the one the server sent.	`SHA256SUMS` in the zip, and the zip's checksum on the download page.

Check the files after unzipping

```
cd gcr-pipeline-samples
sha256sum -c SHA256SUMS          # Linux
shasum -a 256 -c SHA256SUMS      # macOS
```

11. FAQ and troubleshooting

Find the message you see, then follow the steps. The messages are shown exactly as the scripts print them.

The pipeline says the request was refused before it reached the API

The message looks like this:

```
Git Code Review API: HTTP 403: refused before it reached the API.  The runner's IP
is probably not allowed through the load balancer.
```

What it means: Your server only accepts connections from approved IP addresses, and the computer running the pipeline is not one of them.

What to do:

1. Find the runner's public IP address. Add this step to the workflow, run it once, and read the output: `run: curl -s https://api.ipify.org`
2. Send that address to your Git Code Review administrator and ask them to allow it.

3. If the runner is GitHub-hosted (`ubuntu-latest`), its address changes every run. Ask for a self-hosted runner inside your network, or GitHub larger runners with a static IP, and set the `GCR_RUNNER` variable to its label.
4. Run the workflow again.

The token was refused

The message looks like this:

```
Git Code Review API: HTTP 401: the token was refused.  It may be wrong, expired,
revoked, or missing the scope this call needs.
```

What it means: The server did not accept the token.

What to do:

1. On the IDE tokens page, check the token is listed, has not expired, and has the `reports` scope.
2. Check the GitHub secret is named exactly `GCR_TOKEN`, in capitals.
3. If unsure, make a new token and paste it into the secret again. Copy the whole value, starting with `gcr_`.

No token

The message looks like this:

```
No token.  Set GCR_TOKEN, GCR_TOKEN_COMMAND or GCR_TOKEN_FILE.
```

What it means: The script could not find a token anywhere.

What to do:

1. In GitHub Actions: check the step has `GCR_TOKEN: ${{ secrets.GCR_TOKEN }}` under `env:`, and that the secret exists.
2. Pull requests from forks do not get secrets, by GitHub's design. The check cannot run for them.
3. On your computer: check `.env` has one of the token lines filled in.

No server address

The message looks like this:

```
No server address.  Set GCR_URL, or copy .env.example to .env.
```

What it means: The script does not know where Git Code Review is.

What to do: In GitHub, add the `GCR_URL` variable (Step 2). On your computer, copy `.env.example` to `.env`.

GCR_URL must start with https

The message looks like this:

```
GCR_URL must start with https://.  A token sent over plain http can be read by
anybody on the network.
```

What it means: The address starts with `http://`. The scripts refuse it so the token is never sent unencrypted.

What to do: Change the address to start with `https://`.

Other users can read the settings file

The message looks like this:

```
.env holds a token but other users can read it.  Run: chmod 600 .env
```

What it means: The file holding your token is readable by other accounts on the computer.

What to do: Run the command it shows, `chmod 600 .env`, then try again.

The token command did not print a token

The message looks like this:

```
GCR_TOKEN_COMMAND did not print a token.
```

What it means: The command that decrypts or looks up your token did not work.

What to do:

1. Run `bash bash/gcr-token-store.sh test` to check the encrypted token.
2. If GPG asks for a passphrase and none appears, run `export GPG_TTY=$(tty)` first.
3. If you forgot the passphrase, run `bash bash/gcr-token-store.sh save` again.

The branch has not been scanned

The message looks like this:

```
UNKNOWN: Branch release/2.4 of acme/payments has not been scanned, so there is
nothing to check it against.
```

What it means: Git Code Review has never read this branch, so it cannot say whether it is safe. The check stops rather than pass something unknown.

What to do:

1. Ask an administrator to add the branch to the repository's scan scope.
2. Or check the branch the pipeline deploys is the one you expect. On a pull request the scripts use the branch being merged into.
3. Only if your team agrees: add `--allow-unknown` to let it pass.

No such repository

The message looks like this:

```
Git Code Review API: HTTP 404: No such repository.
```

What it means: Either the name is wrong, or your account cannot see that repository.

What to do:

1. Use the full name, `organization/repository`, spelled exactly as in GitHub.
2. Sign in to Git Code Review as the token's owner and check the repository is on the Repositories page. If not, ask an administrator for access.

No SBOM yet

The message looks like this:

```
Git Code Review API: HTTP 404: No SBOM on file for that repository yet.
```

What it means: The repository has not been through a scan that writes SBOMs.

What to do: Wait for the next scheduled scan, or ask an administrator to run one.

There is already an exception

The message looks like this:

```
Git Code Review API: HTTP 409: There is already an exception on this one.
```

What it means: Somebody already asked for an exception on this finding, or one was approved.

What to do: Look at the finding in Git Code Review. Nothing more is needed.

The reason is too short

The message looks like this:

```
Git Code Review API: HTTP 400: Say a bit more about why this is acceptable, at
least 20 characters.
```

What it means: An exception needs a real reason, so a reviewer can judge it.

What to do: Write a sentence explaining why the risk is acceptable, and try again.

Certificate errors

The message looks like this:

```
could not reach https://gcr.example.com/api/v1: [SSL: CERTIFICATE_VERIFY_FAILED]
```

What it means: The server's certificate comes from a company authority your computer does not know. The scripts never skip this check.

What to do: Ask IT for the company CA file (a `.pem` file), and set `GCR_CA_BUNDLE=/path/to/company-ca.pem` in `.env` or the workflow's `env:`.

Too many calls

The message looks like this:

```
Git Code Review API: HTTP 429: too many calls.  Wait a minute and try again.
```

What it means: Many pipelines called at once. The scripts already retry twice.

What to do: Run it again in a minute. If it happens often, run heavy jobs at different times.

jq or curl is missing

The message looks like this:

```
jq is required.
```

What it means: The bash scripts need two small tools.

What to do: Ubuntu: `sudo apt install jq curl`. macOS: `brew install jq`. Or use the Python script, which needs nothing extra.

Why is there no code fix for most code findings?

What it means: Code advice is shown only when an AI review has read that finding and the code has not changed since. A general tip for a rule is not advice about your code.

What to do: Ask an administrator to run an AI review on the findings that matter to you.

I deleted the secret from the code, but it is still listed

What it means: The secret is still in the git history, where anybody with a copy of the repository can find it.

What to do: Rotate the secret at its provider so the old value stops working. That is what makes it safe. Because the old value is still in the history, the finding can stay listed. Tell your administrator it has been rotated, so they can record that on the finding.

The plan says "a range" under Now

What it means: The file names a version range, such as `^4.17.0`, rather than one exact version. The lock file next to it shows the version actually installed.

What to do: Raise the lower end of the range to the Upgrade to version, then rebuild so the lock file updates.

Can I put the token in the workflow file to save time?

What it means: No. Workflow files are part of the repository, so anybody who can read the code could read the token, and it stays in the history forever.

What to do: Always use the `GCR_TOKEN` repository secret.

12. Getting help

- The API reference tab on the IDE tokens page lists every call, and lets you try each one with your token.
- Your Git Code Review administrator can grant access, allow IP addresses, add branches to scans and run AI reviews.

- When you ask for help, include the exact message and the exit code, and never the token.